

Graphs

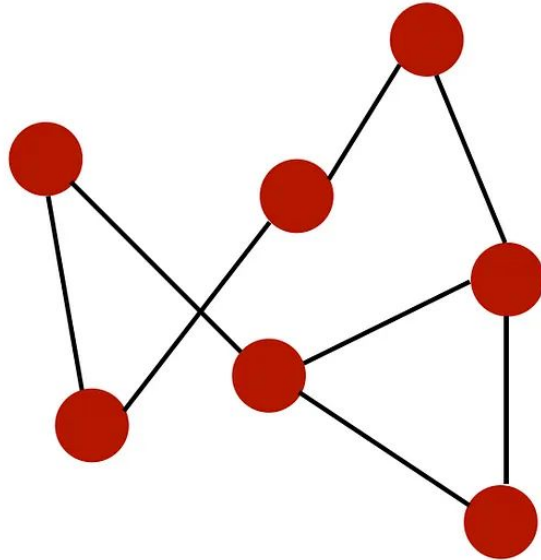
Lecture Flow

- 1) Definition of Graphs
- 2) Types of Graphs
- 3) Graph terminologies
- 4) Graph representations
- 5) Graphs & Trees
- 6) Receiving Inputs on Graph Problems
- 7) Practice questions
- 8) Quote of the day

What is a graph?

Definition

- A way to represent relationships between objects.
- A collection of nodes that have data and are connected to other nodes.



Example: **Friendship Graph**

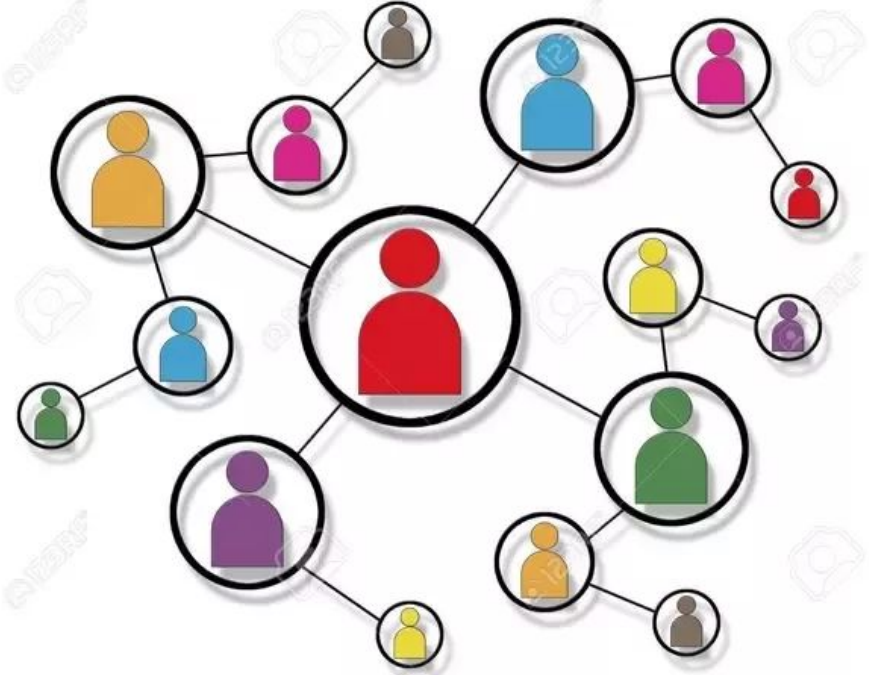
- **Nodes or vertices:**

The objects in graph

- These people
our case

- **Edges:** the relation
between nodes.

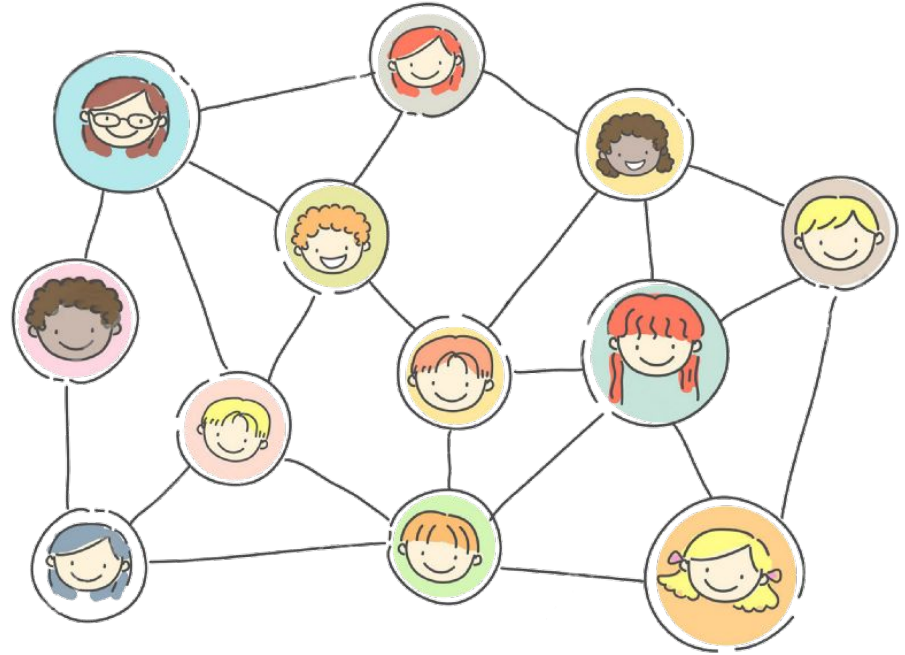
- The friendship
between them
(the lines)



What are the types of graphs?

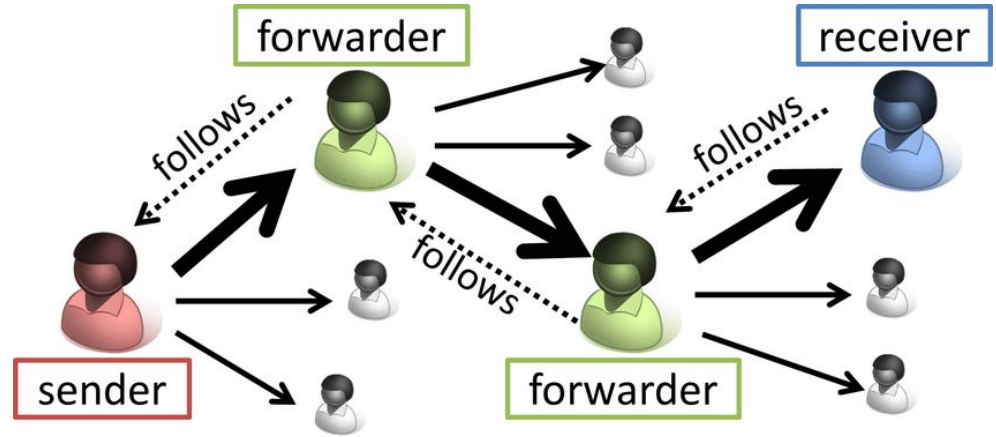
Undirected Graph

- Facebook friendship
- If Alice is friends with Bob, Bob is also friends with Alice



Directed Graph

- Twitter's "following" relation
- If person A follows person B, that does not mean that person B follows person A.

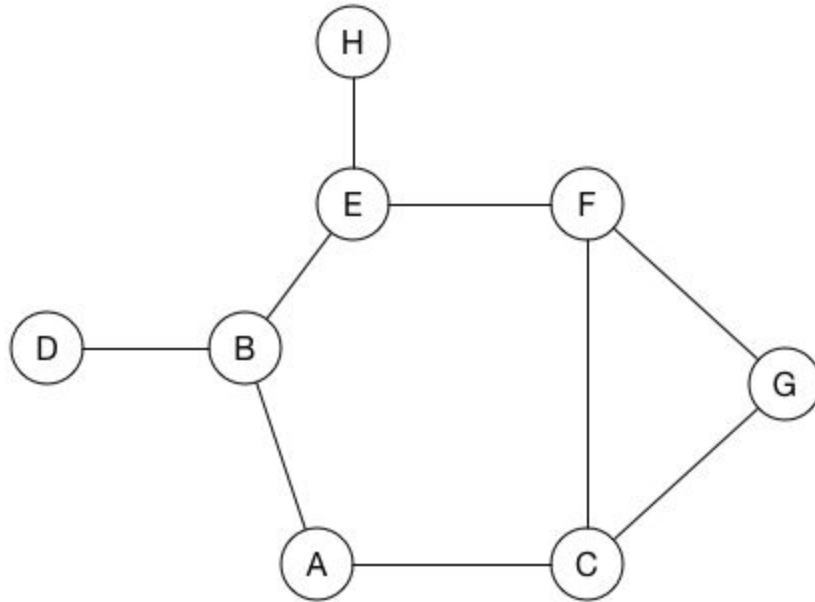


Let's say we want to add a **weight parameter** which represents the **strength of the friendship**.

How can we do that?

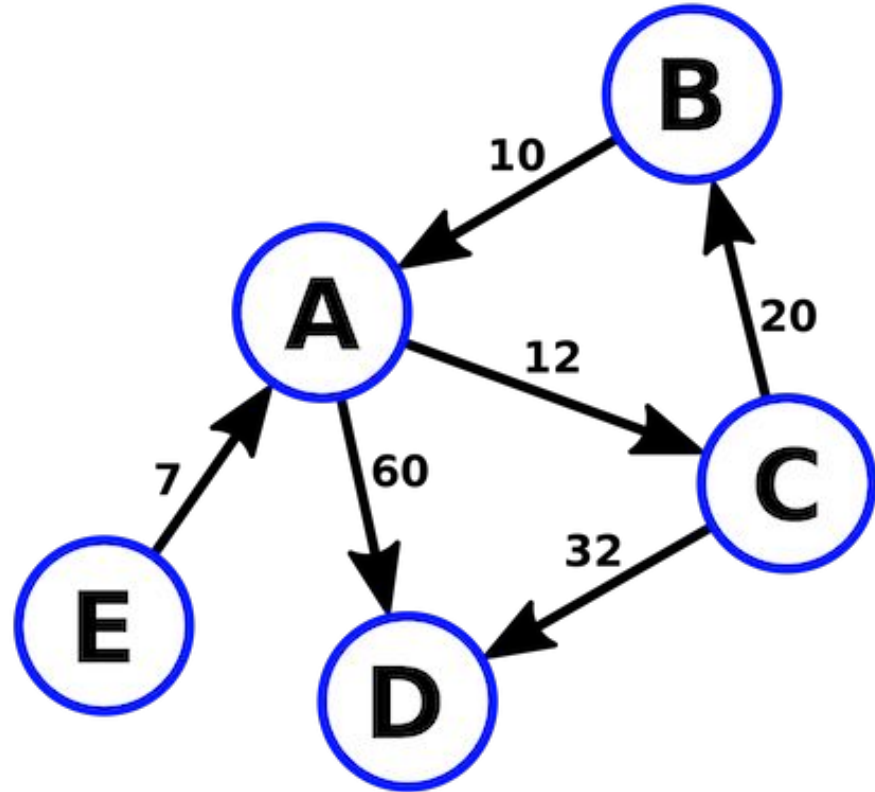
Unweighted Graph

- All edges have the **same value**



Weighted Graph

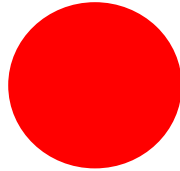
- Weighted graphs assign **numerical values** to edges.



Graph Terminologies

Node / vertex

- Represent entities (e.g., people, devices).
- Connected to other nodes by edges.



Edge

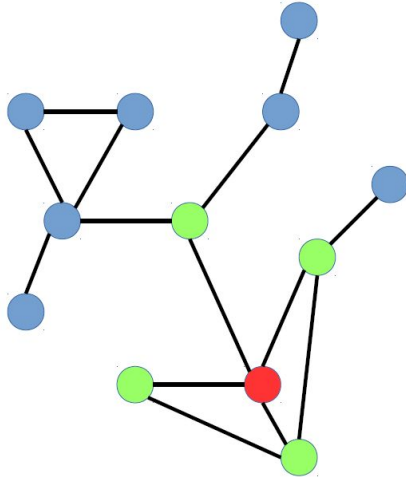
- Connection between two nodes.
- Represented by lines or arrows.
- model relationships in various systems.



Neighbors

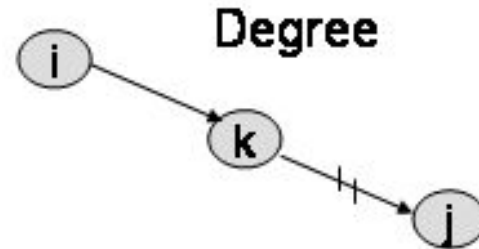
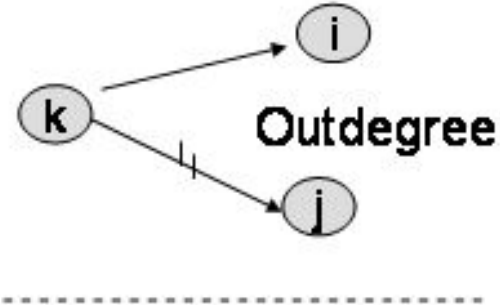
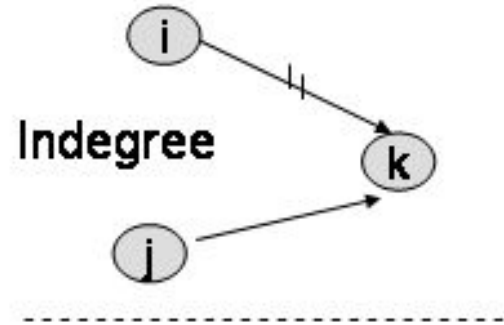
Two nodes are neighbors or adjacent if there is an edge between them

Graph Neighborhood



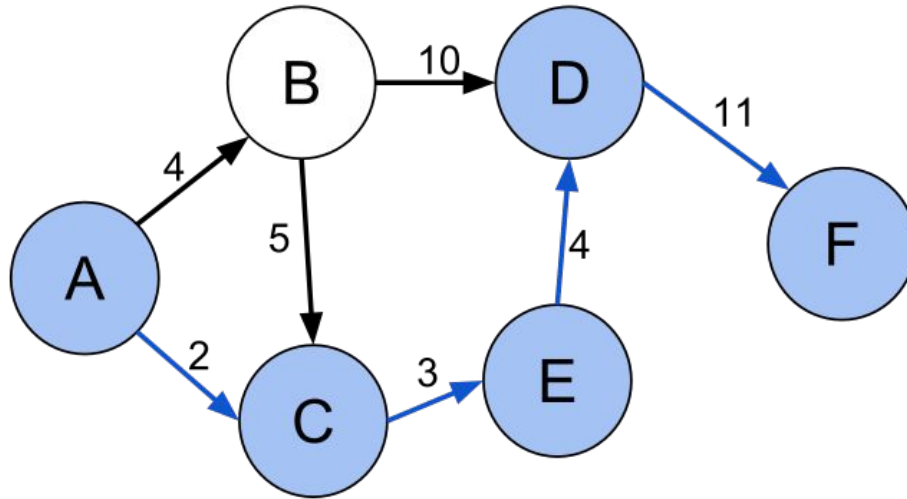
Degree of graph

- Node **degree** = number of neighbors.
- In directed graphs:
 - **Indegree** = edges ending at node.
 - **Outdegree** = edges starting at node.



Path

- A **path** leads from one node to another.
- The length of a path is the number of edges in it.
- Let's consider the path between node A and node F

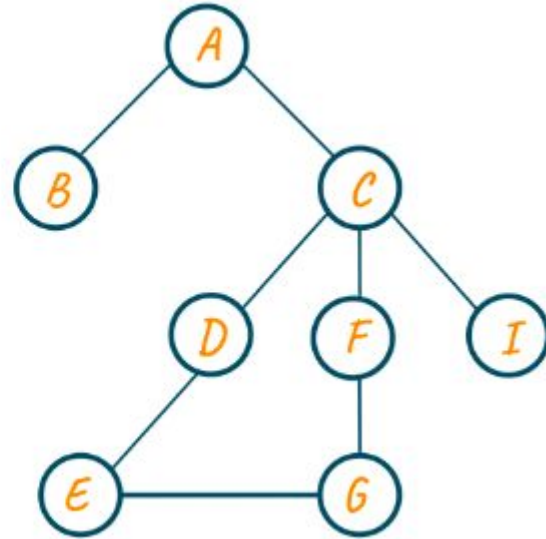


Do you see another path?

Cycle

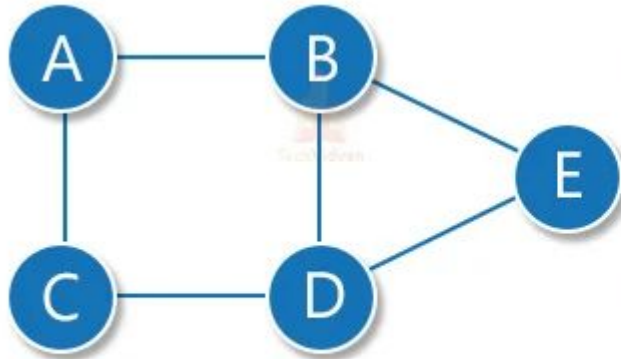
A path is **a cycle** if the first and the last node of the path is the same.

Cycle = C-D-E-G-F-C

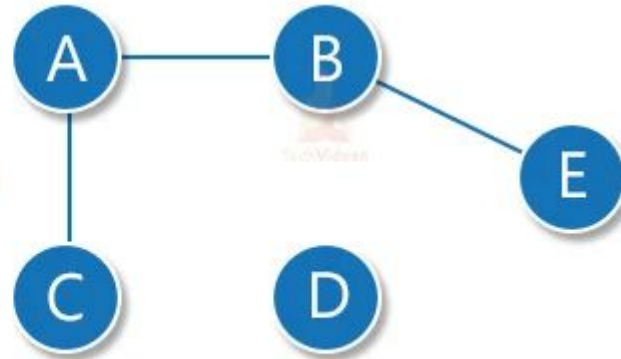


Connectivity

- A graph is **connected** if there is a path between any two nodes



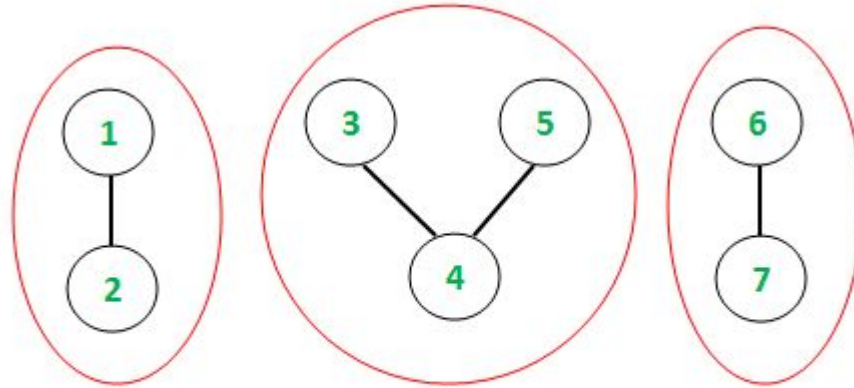
Connected graph



Disconnected graph

Components

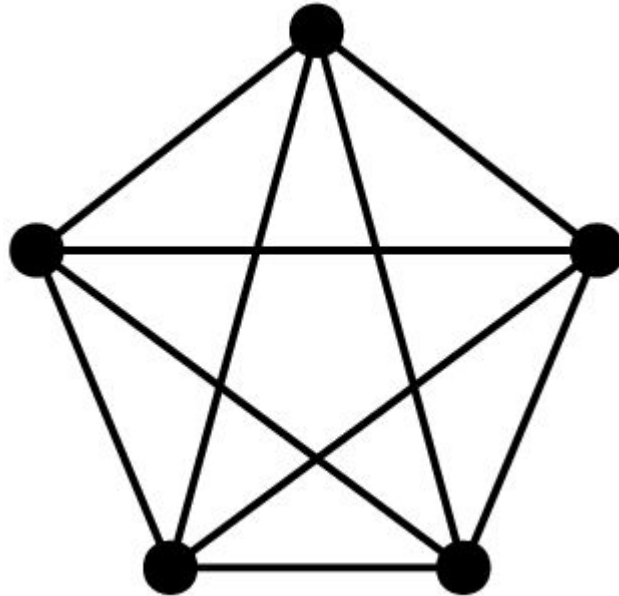
The connected parts of a graph are called its **components**



The **counts** of connected components are - 2, 3 and 2

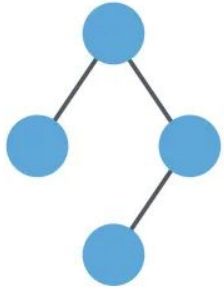
Complete Graph

A complete graph is a graph in which **each pair of node** is connected by an edge.

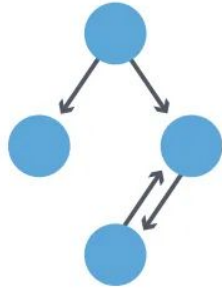


Summary

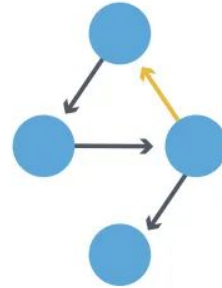
Undirected



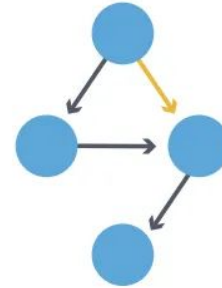
Directed



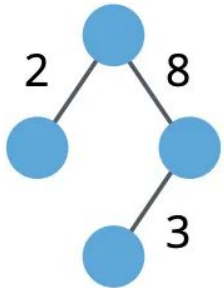
Cyclic



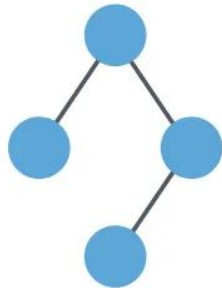
Acyclic



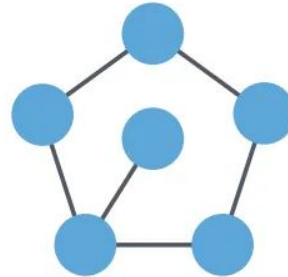
Weighted



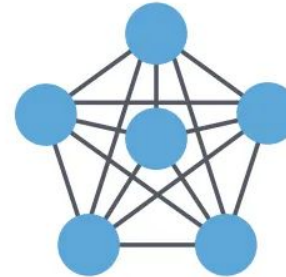
Unweighted



Sparse



Dense



Common Terminologies

Node: _____?

Edge: _____?

Path: _____?

Cycle: _____?

Connectivity: _____?

Components: _____?

Common Terminologies

Tree: _____?

Neighbours(adjacent): _____?

Degree: _____?

Indegree: _____?

Outdegree: _____?

Complete Graph: _____?

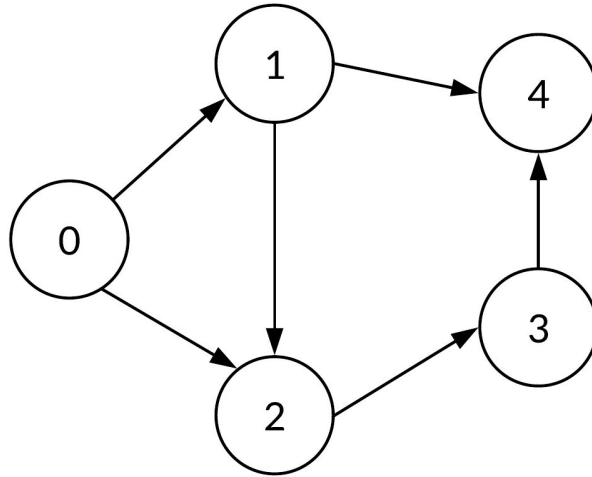
Traverse: _____?

Different ways of representing graphs

Graph Representation

- Adjacency Matrix
- Adjacency List
- Edge List
- Grids as Graphs

Adjacency matrix



Adjacency Matrix

	0	1	2	3	4
0	0	1	1	0	0
1	0	0	1	0	1
2	0	0	0	1	0
3	0	0	0	0	1
4	0	0	0	0	0

Advantages and disadvantages of
adjacency matrix?

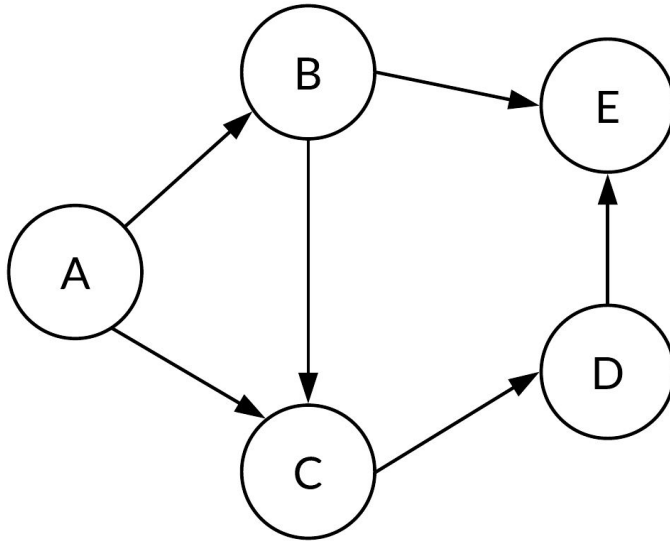
Advantages:

- To represent dense graphs.

Disadvantage:

- It takes more memory ($O(N^2)$)
- Finding neighbors of a node is costly

Adjacency list using list



Adjacency list:

```
graph = {  
  'A': ['B', 'C'],  
  'B': ['C', 'E'],  
  'C': ['D'],  
  'D': ['E'],  
}
```

Advantages and disadvantages of Adjacency List?

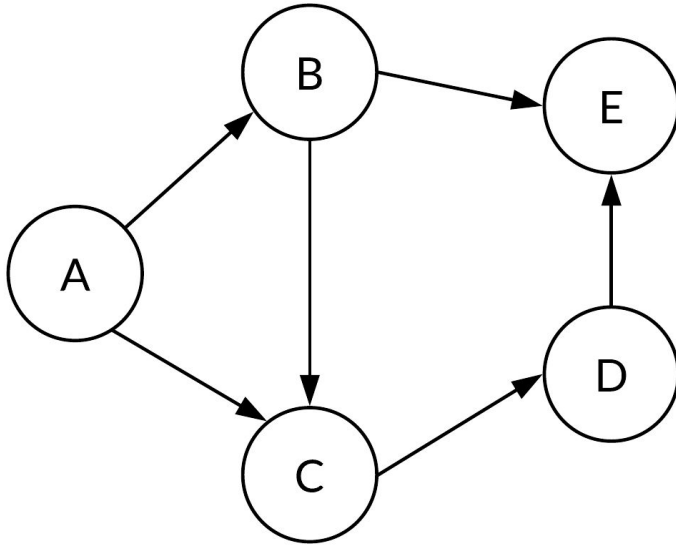
Advantages:

- It uses less memory.
- Best for sparse graphs

Disadvantages:

- Edge look up is slow

Edge List



Edge list:

```
graph = [('A', 'B'),  
         ('A', 'C'),  
         ('B', 'C'),  
         ('B', 'E'),  
         ('C', 'D'),  
         ('D', 'E')  
]
```

Advantages and disadvantages of **Edge List?**

Advantages:

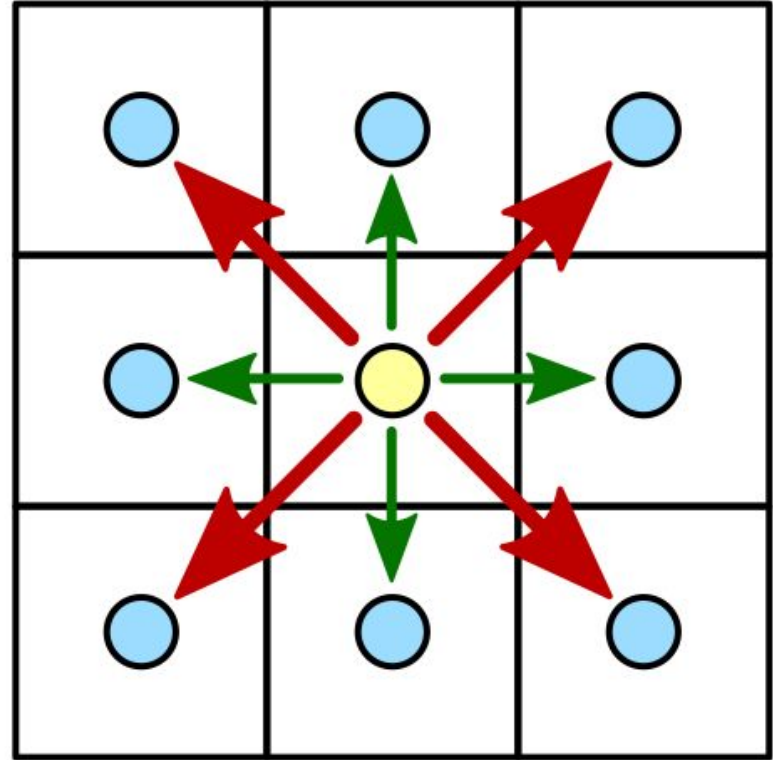
- It uses less memory.
- Easy to represent

Disadvantages:

- Edge look up is slow
- Hard to traverse

Grids as Graph

- Matrix cells = nodes
- Edges between adjacent cells:
 - **4 perpendicular**
 - **2 diagonal/antidiagonal.**



The standard square neighborhood

$(-1, -1)$	$(-1, 0)$	$(-1, +1)$
$(0, -1)$		$(0, +1)$
$(+1, -1)$	$(+1, 0)$	$(+1, +1)$

Only vertical and horizontal neighbors

	$(-1, 0)$	
$(0, -1)$		$(0, +1)$
	$(+1, 0)$	

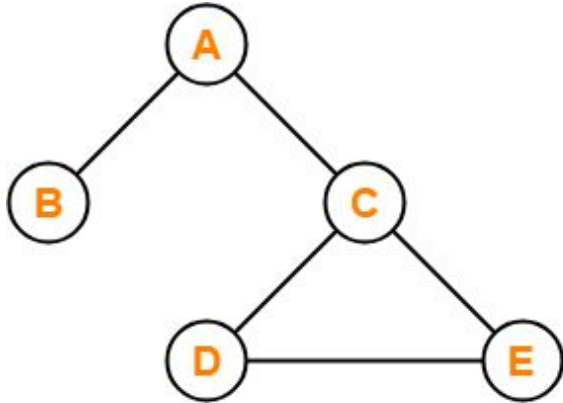
The chess knight neighborhood

	$(-2, -1)$		$(-2, +1)$	
$(-1, -2)$				$(-1, +2)$
$(+1, -2)$				$(+1, +2)$
	$(+2, -1)$		$(+2, +1)$	

Graph and Tree

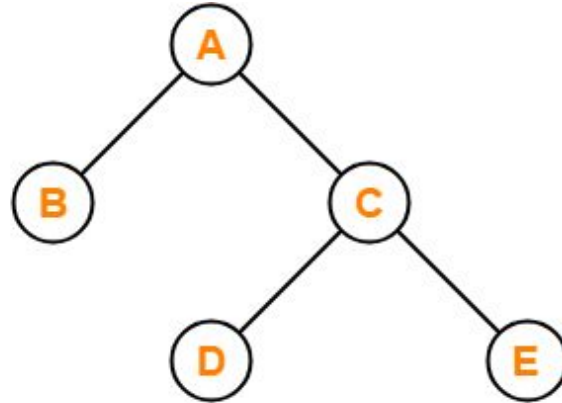
Tree

- A tree is a connected and acyclic graph.
- A tree has a unique path between any two vertices.
- How many edges does a tree have?



X

This graph is not a Tree



✓

This graph is a Tree

Receiving Inputs on Graph Problems

Adjacency Matrix Inputs

On Directed Weighted Graphs

Input:

3

0 1 2

3 4 3

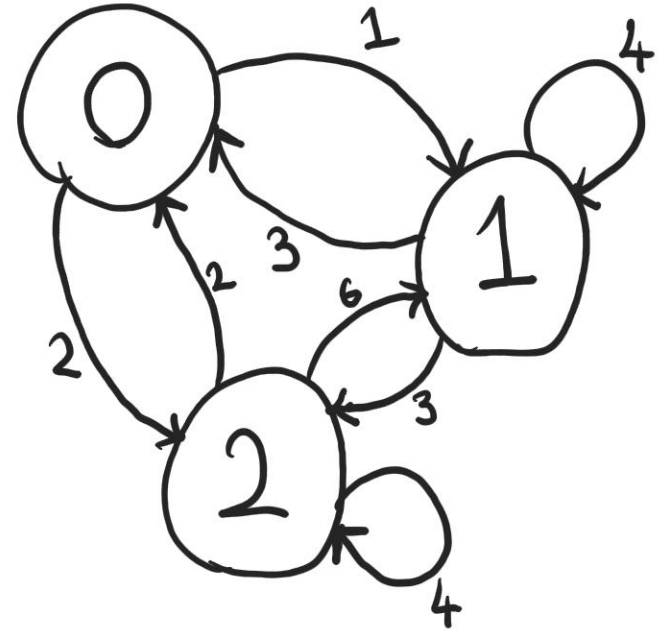
2 6 4

Input:

	0	1	2	
0	0	1	2	
1	3	4	3	
2	2	6	4	

Annotations:

- Matrix Size ($N \times N$)
- Indices (Our Nodes)
- First Row
- Weight between Node 1 and Node 2



Code

```
from collections import defaultdict

n = int(input())
graph = defaultdict(list)

for i in range(n):
    row = list(map(int, input().split()))

    for j in range(len(row)):
        graph[i].append((j, row[j]))
```

```
from collections import defaultdict

n = int(input()) # Matrix size

graph = defaultdict(list)

for i in range(n):
    row = list(map(int, input().split()))

    for j in range(len(row)):
        graph(i).append((j, row[j]))

# Do something with the graph
```

Complexity Analysis

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$

n = number of nodes(matrix length)

On Undirected Weighted Graphs

Input :

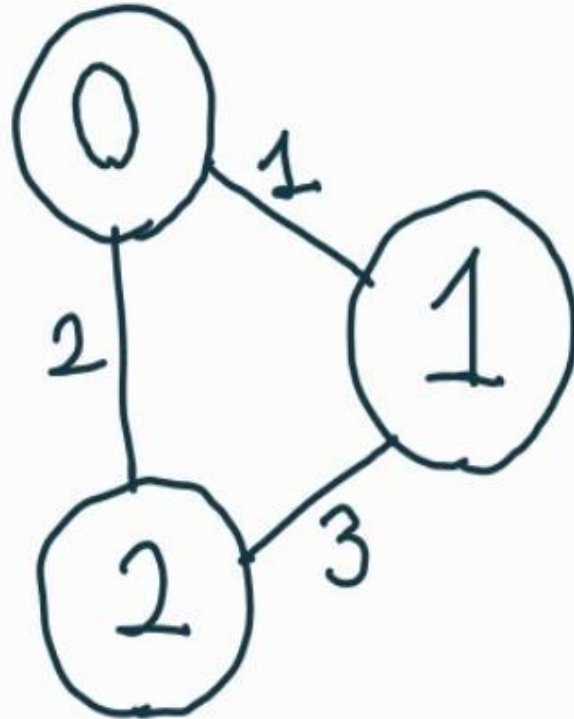
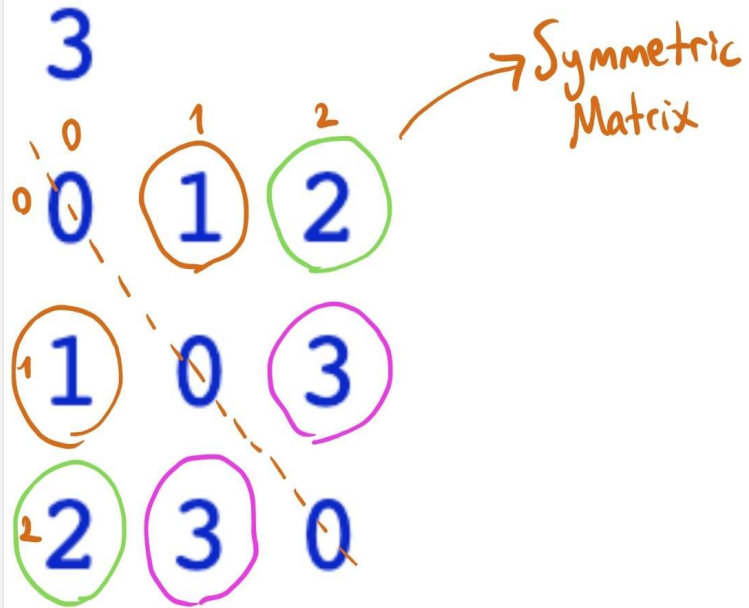
3

0 1 2

1 0 3

2 3 0

Input :



Code

```
from collections import defaultdict

n = int(input())
graph = defaultdict(list)

for i in range(n):
    row = list(map(int, input().split()))

    for j in range(len(row)):
        graph[i].append((j, row[j]))
        graph[j].append((i, row[i]))
```

```
from collections import defaultdict

n = int(input()) # Matrix size

graph = defaultdict(list)

for i in range(n):
    row = list(map(int, input().split()))

    for j in range(i + 1, len(row)):
        graph[i].append((j, row[j]))
        graph[j].append((i, row[j]))

# Do something with the graph
```

Complexity Analysis

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$

n = number of nodes(matrix length)

Edge List Inputs

Input :

3

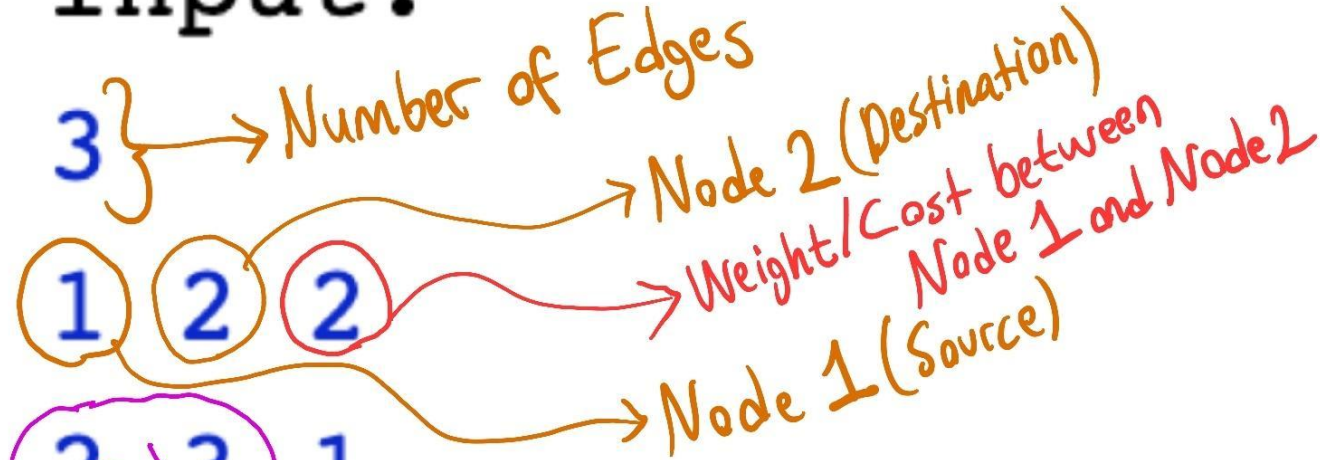
1 2 2

2 3 1

4 1 4

Edge List Input on Directed Weighted Graphs

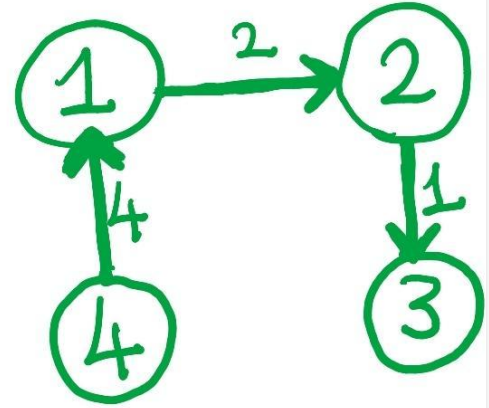
Input:



2 → 3 1

4 1 4

→ This indicates there is an edge from node 2 to node 3 with the cost of 1



Code

```
from collections import defaultdict
```

```
n = int(input())
```

```
graph = defaultdict(list)
```

```
for i in range(n):
```

```
    Src, dest, w = list(map(int, input().split()))
```

```
    Graph[src].append((dest, w))
```

```
from collections import defaultdict

n = int(input()) # Number of Edges

graph = defaultdict(list)

for i in range(n):
    src, dest, w = list(map(int, input().split()))

    graph[src].append((dest, w))

# Do something with the graph
```

Complexity Analysis

Time Complexity: $O(n)$

Space Complexity: $O(n)$

n = number of edges

Adjacency List Inputs

For Undirected Unweighted Graph

Input:

```
[[1, 2], [0], [], [2, 0]]
```

Input:

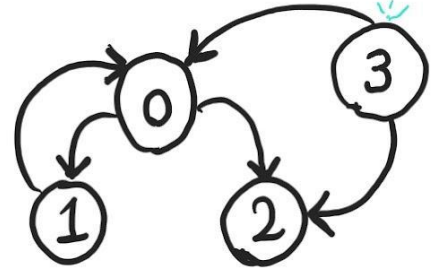
$[[1, 2], [0], [], [2, 0]]$

Children/Neighbours of
Node 0

ith Node

ith Node's
Child

No child
here :-)



Think of ways to implement this

On Directed Weighted Graph

Input:

4

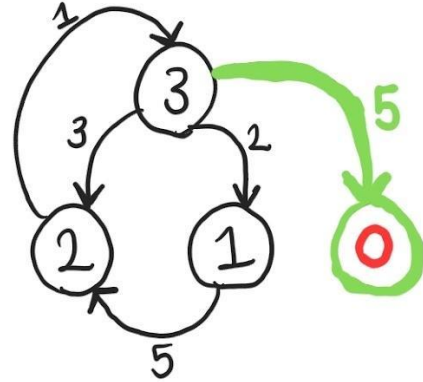
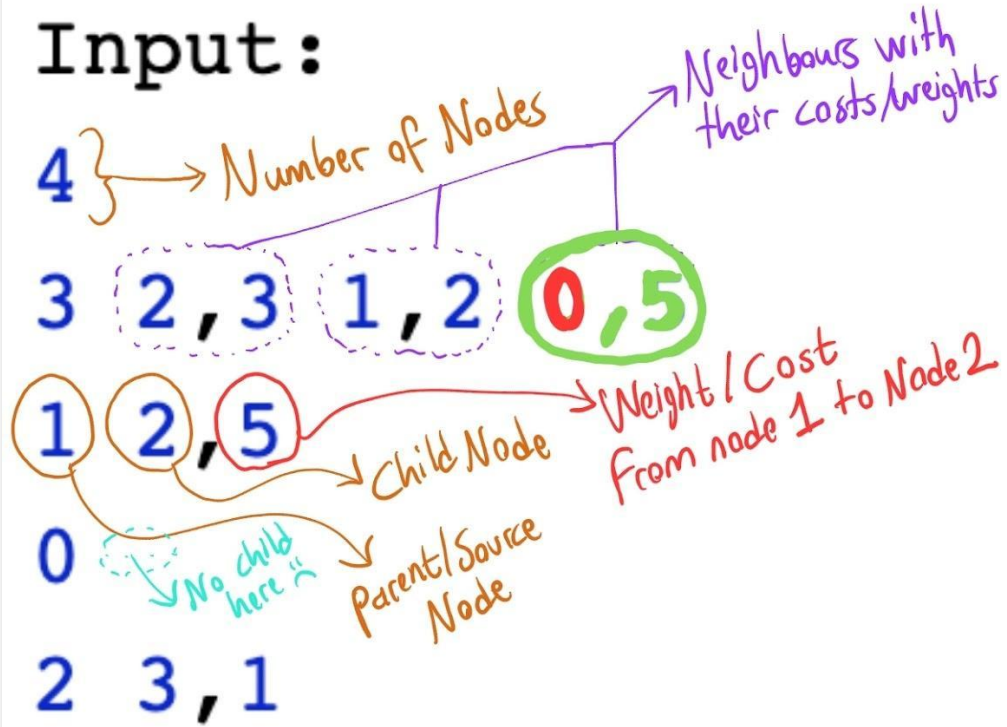
3 2, 3 1, 2 0, 5

1 2, 5

0

2 3, 1

Input:



Code

```
from collections import defaultdict

n = int(input())
graph = defaultdict(list)

for i in range(n):
    line = input().split()
    node = int(line[0])

    for neighbor in range(1, len(line)):
        adj_node, weight = map(int, line[neighbor].split(","))

        graph[node].append((adj_node, weight))
```

```
from collections import defaultdict

n_nodes = int(input())

graph = defaultdict(list)

for _ in range(n_nodes):
    line = input().split()

    node = int(line[0])

    for neighbour in range(1, len(line)):
        adj_node, weight = map(int, line[neighbour].split(", "))

        graph[node].append((adj_node, weight))
```

Complexity Analysis

Time Complexity: $O(n+m)$

Space Complexity: $O(2m)$

n = number of nodes

m = number of edges

For Multiple Graph Problems

```
t = int(input()) # number of test cases

for i in range(t):

    # number of nodes and edges
    n, m = map(int, input().split())

    graph = [[] for i in range(n)]

    for j in range(m):
        u, v = map(int, input().split())

        graph[u-1].append(v-1)
        graph[v-1].append(u-1)

    # do something with the graph
```

Which kind of input is the graph?

OR

```
from collections import defaultdict

t = int(input()) # number of test cases

for i in range(t):

    # number of nodes and edges
    n, m = map(int, input().split())

    graph = defaultdict(list)

    for j in range(m):
        u, v = map(int, input().split())

        graph[u].append(v)
        graph[v].append(u)

    # do something with the graph
```

To make your inputs faster...

```
import sys  
input = sys.stdin.readline
```

This replaces the `input()` function with `sys.stdin.readline()` which is faster.

Common Pitfalls

Common Pitfalls

- Not considering cycles in the graph.
- Not checking whether the graph is directed or undirected
- Not understanding input format well
- Falling in to infinite loop (because of cycles)

Types of Graph Questions

- Graph questions can be classified into different categories based on the problem requirements.
- Some common types of graph questions include:
 - **Shortest path**: find the shortest path between two vertices.
 - **Connectivity**: determine if there is a path between two vertices.
 - **Cycle detection**: detect cycles in the graph.
 - **Topological sorting**: order the vertices in a directed acyclic graph.

Approaches to Solving Graph Problems

- There are several approaches to solving graph problems, including:
 - **Breadth-first search (BFS)**
 - **Depth-first search (DFS)**
 - **Dijkstra's algorithm**
 - **Bellman-Ford algorithm**
 - **Kruskal's algorithm**
 - **Floyd-Warshall algorithm**

The choice of algorithm depends on the **problem requirements**.

Exercise problems...

1. [Operations on graph](#)
2. [Cities and roads](#)
3. [From adjacency matrix to adjacency list](#)
4. [From adjacency list to adjacency matrix](#)
5. [Regular graph](#)
6. [Sources and sinks](#)

For more graph representation Problems: [Link](#)

"There are no mistakes in life, only lessons."

— Robin Sharma

